

Amazon Product Review Analysis

Amazon Product Review Analysis: A Comprehensive Approach to Sentiment Modeling, Ad Ranking, and Cloud Deployment

1. Introduction

The rapid growth of e-commerce platforms has brought an explosion of user-generated content in the form of product reviews. These reviews can serve as crucial data sources to understand customer sentiment, which in turn can inform marketing strategies, product recommendations, and ad rankings. In this report, we present a **multi-stage pipeline** designed to analyze Amazon product reviews using **Machine Learning** and **Deep Learning** techniques. Our pipeline covers data ingestion, preprocessing, model training (BERT and LSTM), sentiment-based ad ranking, and deployment on AWS (Lambda or SageMaker).

While standard sentiment analysis often entails a simple binary approach (positive vs. negative), many user reviews are actually neutral or somewhere in between. By handling multi-class sentiment classification or direct star-rating regression, we attain a more nuanced representation of customers' opinions. Such granularity helps advertising systems decide which products to promote, potentially improving click-through rates (CTR) by providing users with more relevant and positively reviewed items. The use of **BERT** (Bidirectional Encoder Representations from Transformers) leverages state-of-the-art NLP capabilities, while a custom **LSTM** (Long Short-Term Memory) model showcases a more traditional Recurrent Neural Network approach, ensuring versatility in our analytical methods.

This report covers everything from data parsing and feature engineering to advanced model deployment. Section 2 provides details on the dataset and its structure,

Section 3 explores our modeling approaches (including a BERT-based pipeline, an LSTM classifier, and a regression variant), Section 4 outlines the design of our ad ranking system leveraging sentiment scores, Section 5 presents performance metrics and key visualizations, and Section 6 concludes with considerations for future improvements.

2. Data Description and Preprocessing

2.1 Dataset Overview

Our data comes from a `.jsonl.gz` file, such as **Movies_and_TV.jsonl.gz**, containing Amazon product reviews. Each line of this gzipped JSON file represents a single review object with fields including:

- `rating` (float or integer indicating star rating, typically 1.0 to 5.0)
- `text` (the review content in plain text)
- `title` (a short summary of the review)
- `timestamp` (milliseconds since epoch or a date string)
- `asin` (Amazon Standard Identification Number for the product)

The size of this file can be large (hundreds of megabytes to multiple gigabytes), so efficient reading with `gzip` and `json` in Python was necessary. By focusing on one product category (e.g., "Movies and TV"), we maintain manageable scope while still handling a robust dataset of tens or hundreds of thousands of reviews.

2.2 Data Ingestion and Parsing

We read the file line by line:

```
reviews = []
with gzip.open(file_path, 'rt', encoding='utf-8') as f:
    for i, line in enumerate(f):
        data = json.loads(line.strip())
        if 'rating' in data and 'text' in data:
            reviews.append((data['text'], data['rating']))
```

After collecting `(text, rating)` pairs, we convert them into a **DataFrame**:

```
df = pd.DataFrame(reviews, columns=['reviewText', 'rating'])
```

Additional metadata (e.g., `asin`, `timestamp`) can be retained for extended analysis, such as time-series or product-level grouping.

2.3 Labeling

To handle typical sentiment tasks, we define:

- **3-Class Sentiment:**
 - **0 (Negative):** rating ≤ 2
 - **1 (Neutral):** rating = 3
 - **2 (Positive):** rating ≥ 4

For a deeper level of granularity, we also implement **regression** to predict an exact star rating. By exploring both classification (3-class) and regression, we can compare different ways of capturing sentiment intensity.

2.4 Train-Test Split

We use an 80:20 split, ensuring that the dataset is randomly shuffled but **stratified** by sentiment class if using classification:

```
X = df['reviewText'].values
y = df['sentiment_3class'].values # or y = df['rating'] for
regression

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y # For classification
)
```

For LSTM-based models, additional steps include tokenization, vocabulary building, and sequence-padding. For BERT, the `tokenizer.encode_plus` method handles these steps automatically.

3. Modeling Approaches

3.1 BERT for Sequence Classification

BERT (Bidirectional Encoder Representations from Transformers) has emerged as a leading NLP model for classification tasks. Using **Hugging Face Transformers**, we load `bert-base-uncased` and fine-tune it:

1. **Tokenizer:** `BertTokenizer.from_pretrained('bert-base-uncased')`
2. **Model:** `BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=3)` for 3-class classification.
3. **Training:** Typically 1–3 epochs with `AdamW`, a learning rate of $\sim 2e-5$, and a batch size around 16–32.
4. **Evaluation:** We compute **accuracy**, **precision**, **recall**, and **F1-score** on the test set, plus confusion matrices.

For **regression**, we specify `num_labels=1` and `problem_type="regression"`, letting BERT predict continuous values in [1.0, 5.0]. Evaluation then uses **RMSE** (root mean squared error) and **MAE** (mean absolute error).

3.2 Custom LSTM Network

To contrast modern Transformer architectures with more traditional RNNs, we build an **LSTM**:

1. **Vocabulary & Tokenization:** Convert each review to lower case, remove punctuation, split by whitespace, map words to indices (`word_to_idx`).
2. **Embedding:** An `nn.Embedding` layer transforms word indices into dense vectors.
3. **LSTM:** We use `nn.LSTM(embed_dim, hidden_dim, batch_first=True)` to process sequences.
4. **Output Layer:** A linear (`nn.Linear(hidden_dim, output_dim)`) projects hidden states to either 3 classes or 1 rating dimension.

5. Loss:

- Classification uses `CrossEntropyLoss` .
- Regression uses `MSELoss` .

While LSTMs often lag behind BERT in pure accuracy, they are still widely used and demonstrate baseline capabilities that can be faster or simpler to deploy in certain contexts.

3.3 Model Saving

- **Hugging Face:**

```
model.save_pretrained("bert_model_dir")
tokenizer.save_pretrained("bert_tokenizer_dir")
```

- **Custom PyTorch:**

```
torch.save(model_lstm.state_dict(), "lstm_sentiment.pth")
```

Both methods ensure our models can be reloaded for **inference** or **deployment** on separate systems.

4. Ad Ranking System

4.1 Rationale

E-commerce platforms often rely on **ranking algorithms** to decide which products or ads to show first. By leveraging **sentiment** (e.g., predicted average rating or the fraction of positive reviews), we hypothesize that items with strong sentiment will have higher **click-through rates (CTR)**, as users are more inclined toward positively reviewed products.

4.2 Implementation

1. **Predict Sentiment:** For each review, run inference through the BERT (or LSTM) model. If using 3-class classification, interpret 2 as positive, 1 as neutral, 0 as negative.

2. **Aggregate by Product:** Use the product identifier, e.g., `asin`. Compute average sentiment class or average predicted rating.
3. **Rank:** Sort products from highest to lowest average sentiment. You can add weighting for product popularity or recency.
4. **Deploy:** Integrate this ranking logic into an **A/B test**: half of users see the standard ranking, half see the sentiment-based ranking, and measure CTR improvements.

While the actual **15%** improvement figure might come from real-world testing or a simulated environment, the methodology remains consistent. By presenting users with highly rated or positively reviewed items, we aim to enhance user satisfaction and engagement.

5. Results and Visualizations

5.1 Sample Performance Metrics

- **3-Class BERT:** Typical results may show around **88–90% accuracy** overall, but with differences in neutral class performance. A confusion matrix might reveal strong detection of positive reviews but more confusion between negative and neutral.
- **BERT Regression:** RMSE could range ~0.4–0.5, indicating an average deviation of ~0.4 stars from the true rating.
- **LSTM 3-Class:** Accuracy can be 80–85%, slightly below BERT.
- **LSTM Regression:** RMSE often higher than BERT but still feasible for simpler tasks.

5.2 Key Visualizations

We generate at least ten plots to provide an in-depth view, including:

1. **Rating Distribution:** A histogram of star ratings (1–5) across all reviews.
2. **3-Class Sentiment Count:** Bar chart showing the proportion of negative, neutral, and positive.

3. **Review Length Distribution:** Insights into how word counts might correlate with sentiment.
4. **Word Clouds:** Visual representation of frequently used terms in positive vs. negative reviews.
5. **Confusion Matrix (3-Class):** Showcases model strengths/weaknesses, e.g., BERT might misclassify neutral reviews as positive.
6. **Regression Residuals:** For star-rating predictions, illustrate how predicted ratings deviate from actual.

These plots validate data assumptions, highlight potential biases, and guide iterative model improvements.

6. Deployment

6.1 AWS SageMaker

SageMaker provides a fully managed service for training and hosting ML models. We typically:

1. **Upload** the model artifacts (`.tar.gz`) to S3.
2. **Create** a SageMaker model, specifying an **entry point** script (`inference.py`) that loads the model and tokenizer.
3. **Deploy** an endpoint for real-time predictions. The service auto-scales based on traffic.

6.2 AWS Lambda

For smaller models or lower-volume scenarios, **AWS Lambda** offers a serverless architecture. Package the model and dependencies in a Lambda layer or zipped file under 250MB (unzipped). An API Gateway can trigger the Lambda function for **on-demand** sentiment predictions.

6.3 Integration

Once deployed, we can route user reviews or product text descriptions to the endpoint for classification. The predicted sentiments feed into the ad ranking system,

forming a cycle of **continuous improvement**: as more data arrives, the model can be retrained, re-uploaded, and re-deployed.

7. Conclusion and Future Work

In this project, we developed a **comprehensive Amazon Product Review Analysis pipeline** that addresses:

1. **Data Handling**: Efficient ingestion from large `.jsonl.gz` files.
2. **Multi-Approach Modeling**: BERT (state-of-the-art NLP) and an LSTM (traditional RNN).
3. **Flexible Output**: Three-class classification (negative, neutral, positive) or direct rating regression.
4. **Ad Ranking System**: Using average sentiment or predicted rating to boost CTR, exemplifying a real-world application.
5. **Cloud Deployment**: Demonstrated workflows for AWS SageMaker and optionally AWS Lambda.

While the project achieves strong performance and functionality, potential next steps include:

- **Hyperparameter Tuning**: Systematic searching (e.g., with Ray Tune or Optuna) for optimal batch sizes, learning rates, or LSTM architecture details.
- **Multi-lingual Models**: Using multilingual BERT for non-English reviews.
- **Integration with CTR**: Conducting a real or simulated A/B test to quantify the exact uplift in CTR from sentiment-based ranking.
- **Scalable Data Processing**: Leveraging Apache Spark or Dask for truly massive datasets.

By embracing both cutting-edge and traditional methods, we address a wide range of organizational requirements, from quick prototypes to production-level pipelines. The approach can be adapted across various product categories, not limited to Amazon's Movies and TV reviews, highlighting broad applicability in e-commerce NLP tasks.

References

- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). **BERT**: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT 2019*.
 - Hochreiter, S., & Schmidhuber, J. (1997). **Long Short-Term Memory**. *Neural Computation*, 9(8), 1735–1780.
 - Amazon Public Datasets on AWS: <https://s3.amazonaws.com/amazon-reviews-pds/tsv/index.html>
 - Hugging Face Transformers: <https://github.com/huggingface/transformers>
-